Teszári Gergely (UBMX0L) Programozói specifikáció a félkész Nagy Házi feladathoz

2020, BPROF Progrmozás Alapjai

Ami eddig megvan... hibásan, vagy működőképesen

Eddig működik:

Függvény:

- Clear() (clear.py)
- Error() (error.py)
- Start() (NagyHazi_2020.py)
- MainMenu() (NagyHazi_2020.py)
- TableOpener (asztalok.py)
- MenuOpener (etelek.py)
- Table_opening (asztalok.py)
- Table_modify (asztalok.py)
- Modify() (asztalok.py)

Osztály:

- Table (asztalok.py)
    - __str__()
    - save()
- Menu (etelek.py)
    - __str__()
    - save()

Leírás

(Könnyűtől haladva az egyre bonyolultabb felé)

clear.py:

Egy menüvezérelt programban fontos, hogy ne egymás alá kerüljenek a kiírt menük és felhasználói felületek. A konzolt minden művelet után törölni kell, ezt a törlést hivatott a **Clear()** függény megtenni, ami mintegy kis kiegészítő függvény, minden indításkor meghívásra kerül a menü által. Az egyetlen, amit csinál, hogy törli mindazt, ami a konzol felületén addig írva volt.

error.py:

Ez a fájl egyetlen függvényt tartalmaz, a nagybetűs Error-t, melynek egyetlen paramétere az id. Ez a fájl és függvény kizárólag a hibakezelésre van, a többi függvény hiba esetén meghívja, megadja neki, milyen típusú hibáért hívta, és visszatér. Maga az **Error()** mindössze annyit tesz, hogy szépen kiírja, milyen típusú a hiba, vár 2 másodpercet, és visszaadja a szót annak, aki meghívta.

NagyHazi_2020.py:

Ez a fálj a lelke az egész programnak. Ez két dolgot tartalmaz, mint tevékenység: egy kis teknőcgrafikás bootup logo rajzoló függvényt, ez a **Start()**. Ez hívódik meg mindenek előtt. Ha végzett a logó kirajzolásával, vár 2 másodpercet, és meghívja a **MainMenu()** függvényt, ami az összes folyamatot vezérli. Ez a menüvezérelt program menüje, ami kiírja a lehetséges opciókat, amiket a programmal el lehet végeztetni. Egy végtelen loop, ami kizárólag akkor lép ki, ha a felhasználó az "e" betűt adja be neki. Ebben az esetben egy break-kel kiszáll a végtelen ciklusból, ami egy while. Az opciók számok, ezek beírásával lehet a továbbiakban tárgyalásra kerülő függvényeket meghívni. Minden egyes választás a choose nevű változóban tárolódik, amit utána megvizsgálunk, ha létező opciót választottak, akkor meghívja az opció mögött álló függvényt, ha egyik opciót sem fedi le az értéke, hibajelzés keletkezik (**error.Error(4)**), miszerint nem létező opciót választottak, majd előlről indul a while ciklus. A menü minden egyes indulásakor és visszatérésekor meghívja a **clear.Clear()**-t, ezzel segítve az áttekinthetőséget. Az utolsó dolog, amiről ebben a fájlban tudni kell, hogy az elején be vannak include-olva az étlap és az asztalok kezelésével foglalkozó függvények, a menüpontok, hogy ne egy fájlban legyen minden. Maga a menü olyan szerepet tölt be, mint egy forgalmi lámpa, jelez, ki indulhat el. Visszatérési értéket, eredményt, semmit sem vár el a meghívottól, ő csak megszólítja, a függvény elvégzi a dolgát, interakcióba kerül a felhasználóval, ennek függvényében kiír dolgokat, majd ha végez, visszaadja a szót a menünek.

asztalok.py:

Innentől kezdődik a projekt érdemi része.

A modul először definiál egy osztályt. Ennek az osztálynak a tulajdonságai: ez egy asztalokkal foglalkozó osztály, ami tartalmazza az asztalok: sorszámát, férőhelyeinek számát, azt, hogy jelenleg foglalt vagy szabad-e az adott asztal, és az elhelyezkedésére vonatkozó x és y koordinátát. Itt szépen meg van szabva, hogy melyik tulajdonság milyen típusú (a foglaltság-mutatót leszámítva mind integer, a foglalságot egy s, mint szabad vagy f, mint foglalt sztring jelzi). Az osztály tartalmaz még egy képernyőre kiíratás **[__str__()**] és egy fájlba kiíratás [**save()**] alfüggvényt, mindkettő a saját szintatikájával működik. (A képernyőre vesszővel, fájlba alulhúzással elválasztott rekordokat ír ki.) Kezeli a beolvasás során keletkezett hibákat, ha az adott elem nem megfelelő típusú, hibát dob, amit az error.Error() segítségével vissza is jelez számunkra.

Kell viszont valami, ami a "keze alá dolgozik", ez az alatta lévő immár függvény, a **TableOpener()**. Ez végzi a fájl (tables.txt) beolvasását, szétdarabolását, osztálynak megfelelő csoportosítását, és az osztály minden paraméterével rendelkező elemek listába rendezését. Ezzel a listával tér vissza. A következő hibákat kezeli: Nem létezik a fájl, vagy az állomány hibás, nem konvertálható át a kívánt osztály elemeivé. Ha ez a két hiba nem áll fenn, akkor sorokat szed ki a állományból, melyeket darabol, és a megfelelő osztály paraméterrel ellátva hozzáfűzi egy listához, ha az elemek típusa stimmel.

Nagyon hasonló, de azért különböző névvel következik utána a **Table_opening()** nevű függvény, ami az asztalnyitásra szolgál. Először is meghívja a TableOpenert, és annak visszatérési listáját eltárolja

magánál, szintén egy listában. Megkérdi a felhasználót, hogy melyik asztalt szeretné lefoglalni, ami döntés meghozatalához segítségképpen kiírja a képernyőre az összes szabad beolvasott asztal sorszámát, és helyeinek számát. Ha a felhasználó nem számot ad meg, vagy akkora számot ad meg, amekkora sorszámmal asztal már nem létezik, hibát dob, az **error.Error()** segítségével. Ha a felhasználó olyan asztalt kíván lefoglalni, amely már foglalt, arra külön figyelmeztet, egy olyan hibaüzenettel, ami az Error() függvényben nem szerepel, hiszen az egész programban csak egyszer fordulhat elő. Ha az előbbi hibák egyike sem áll fenn, akkor a saját kis belső listájában foglaltra állítja az adott asztalt. Ezt követően a beépített save() függvénnyel felülírja az egész tables.txt fájl tartalmát, beleírja a saját kis listáját, a templistet, ennek sikeressége esetén értesíti a felhasználót a foglalás sikerességéről.

Az asztalok kirajzolására szolgál a következő függvény, a **Table_drawing()**. Ez is eltárolja az asztalok listáját a **TableOpener()** függvény meghívásával. Létrehoz egy HD felbontású, téglalap alakú rajzteret, ahová egy teknős fogja berajzolni az asztalokat. Az ablak címe "Az asztalok elhelyezkedése", háttérszíne fekete. Kezdetben a teknős is fekete, hogy amíg a helyére mászik, ne látszódjon, egyébként pedig olyan gyors, amilyen gyors csak lehet. Ezt a teknőst szépen elindítjuk egy for ciklus által vezérelve, amely ciklus szépen végigmegy az asztalokon a kis belső listában, egyenként, az x és y koordinátára elhelyezi a teknőst. Ha az asztal, amit rajzolni készülünk, szabad, a teknőc színét beállítjuk zöldre, ellenkező esetben pirosra. Itt is kezeljük az esetleges hibákat, mert habár a beolvasófüggvény megnézi, hogy sztring -e, de nem nézi, hogy csak kétféle betű lehet ott. Ezután a teknős elkezdi az asztal rajzolását, annak bal felső sarkába gyalogol, és ciklus segítségével négyszer húz vonalat, majd fordul jobbra 90 fokot. Az így keletkezett négyszöget kitölti a megadott színnel, majd beleírja a férőhelyek számát. Ezt eljátsza minden asztalnál.

Az utolsó dolog, amit az asztalokkal kell tudnunk végezni, az a paramétereik módosítása, új asztal hozzáadása, meglévő törlése. Ezt egy összedolgozó függvénypáros, a **Table_modify()**, és a **Modify()** oldják meg. Először az előbb említett függvény hívódik meg. Ez egy kis menüt tár a felhasználó elé, amely az asztalokkal végezhető műveleteket sorolja fel. Ezek közül az adott menüpont beírásával lehet választani, ami után megkérdi a program, hogy melyik asztalon kívánjuk elvégezni a módosítást. Értelemszerűen az asztal hozzáadásánál nem kérdez ilyet, hanem nullára állítja az asztal sorszámát, amin a műveletet el kívánjuk végezni. Itt szűri az esetleges hibákat is: ha nem létező opciót választottunk, vagy ha a kérdéses asztal sorszám olyan nagy, amekkora sorszámú asztal nem is létezik.

Ha minden oké, meghívja a két paraméterrel (mit akarunk csinálni és melyik asztalon) a **Modify()** függvényt. Ez végrehajtja az utasítást, immáron biztosan abban, hogy a kért művelet a megadott asztalon elvégezhető. Ő is eltárolja mindenekelőtt az asztalok listáját, amiből aztán dolgozik. Ha a kért művelet nem a törlés, a két ponttal jellemezhető koordináta módosítása, vagy az új asztal hozzáadása, akkor csak egy paramétert vár, ezt megkérdi, mire szeretnénk módosítani, majd ezt betölti a megfelelő helyre a megfelelő asztalhoz. Itt még le is ellenőrzi, hogy a sorszám helyére számot, esetleg korábban már szereplő számot, a foglalt/szabad helyére f/s betűt próbál-e berakni a felhasználó, ha nem, hibát dob. Ha koordinátákat akarunk cserélni, akkor két értéket kér, megvizsgálja, hogy számokat kapott-e, szerepelt-e már korábban ilyen koordinátával asztal, ha ezek valamelyike fennáll, hibát dob, ha ezek nem állnak fenn, betölti a megfelelő helyre. Ha hozzáadni kívánunk új asztalt, akkor először megnézi, hogy az asztal száma 0-ra van-e állítva, ha nem, hibát dob, ha igen, szép sorban bekéri az új asztal adatait, a sorszámot automatikusan generálja, úgy, hogy végigmegy az eddigi asztalokon, és a legnagyobb sorszámú asztal sorszámához ad egyet, a foglaltságot automatikusan szabadra állítja. A bekért adatokat mind ellenőrzi, szám-e, aminek számnak kell lennie. Megnézi továbbá, szerepelt-e előtte olyan sorszámú asztal, megnézi, hogy az új

asztal x és y koordinátája szerepelt-e már korábban. Ha igen, hibát dob. Ha törölni kívánunk, akkor az adott asztalt törli a saját kis listájából. Ha végzett, a saját belső listáját a **save()** függvény segítségével beleírja a tables.txt-be, felülírva ezzel annak eredeti tartalmát, majd visszajelez, hogy a módosítás sikeres volt.

etelek.py:

A program érdemleges második fele.

Az első két csoport, osztály definiálása [**class Menu**] és a függvény, amely beolvassa a fájlt, és visszatér egy listával [**MenuOpener()**], pontosan ugyanúgy működik, mint az asztaloknál, annyi különbséggel, hogy itt csak két elem van alulhúzással elválasztva a fájlban és definiálva az osztályban, melyek az étel neve, sztring, és annak ára, int típusban. Ugyanolyan hibákat kezel, ugyanaz a felépítése és a működése, csak még egyszerűbb és rövidebb.

Az első, ahol különbséget látunk, az a modul első érdemi függvénye, a rendelés leadására szolgáló **Order()** függvény. Hasonlóan a korábbiakhoz, ez is először eltárolja magának egy kis belső listába az étlapot, a **MenuOpener()** meghívásával. Először a felhasználótól bekéri az asztal számát, ahol a rendelést felvették. Megnézi, hogy számot kapott-e, ha nem, hibát dob. Ezután betölti az asztalokat egy kis belső listába, megnézi, hogy az adott sorszámú asztal létezik-e, ha nem, hibát jelez, foglalt-e, ha nem, foglaltra állítja. Az asztal számát egy változóba írja, és mögé írja a kiterjesztést is: ".txt". Ezután a felhasználó segítésére a korábbi specifikációtól eltérően nem az ételek nevét várja egy ciklusban üres sorig, hanem azok index változóval generált sorszámát, ami sorszámokat utána megvizsgál, hogy szám-e, majd ha ilyen indexű elem létezik a saját kis belső listájában, akkor az azon az indexen álló ételt árával együtt egy Menu típusban hozzáadja egy listához, és kiírja, hogy a hozzáadás sikeres. Ha üres sort kap, ami a bemenet végét jelzi, akkor az így töltögetett lista tartalmát kiírja először a képernyőre, vár egy kicsit, majd **save()** függvény segítségével beírja ezt a listát egy txt fájlba, amit a korábban egy változóban eltárolt módon nevez el asztalszám.txt-nek. Ha ez is kész, még egyszer megerősíti a felhasználót, hogy a rendelés leadása sikeres volt.

Erre épül rá a számla nyomtatása, a **Receipt()** függvény. Ez először bekér egy számot, annak az asztalnak a számát, amelyik asztalhoz tartozó számlát szeretnénk nyomtatni. A bekért értéket megvizsgálja, szám-e, létezik-e ilyen sorszámú asztalhoz számla fájl. Mindkét eset nem teljesülése esetén hibát dob. Beolvassa a fájlt soronként, tördeli a sorokat, alulhúzásonként, kiírja a sorokat, és a végösszeget. A végösszeget úgy kapja, hogy minden egyes szétdarabolt beolvasott sorból az árat hozzáadja a végösszeghez, int-ként. Ha ez nem int lenne, hibát dob. Ezután ha a felhasználó kérésére beír egy "x"-et, törli a fáljt, kiírja, hogy fizetve. Továbbá itt is beimportálja egy belső listába az asztalokat, megnézi, hogy az adott asztal foglalt volt-e eddig, ha igen, szadadra állítja, ha nem, hibát dob, mert olyan asztaltól érkezett rendelés, ami szabad. Ez alapvetően nem egy akkora baj, de azért figyelmeztet.

Ezzel elérkeztünk az utolsó lehetséges és elvégezhető művelethez, amit hasonlóan az asztalokhoz, itt is két egymásra épülő függvény hajt végre. A **Menu_modify()** és a **Modify()** teljesen úgy működnek, mint az asztalos párjuk, csak itt rövidebb, ugye. Annyiban más viszont az egész, hogy itt ugye nem a kokrét elemekkel dolgozunk, hanem az ételeket megindexeljük, és az adott indexű elemen hajtunk végre módosításokat. Ezt aztért teszi így, hogy ne kelljen begépelni a teljes étel nevét hibátlanul. Minden hibát hasonlóan kezel. A hármas és négyes, azaz az új étel hozzáadása és meglévő törlése kivételével mindig egy-egy értéket módosít a megfelelő listaelemben. Ha az étel neve kerül módosításra, megnézi, volt-e már olyan nevű étel, ha volt, hibat dob. Ha az ár a módosítás alatt lévő elem, akkor megnézi, int-e, ha nem, hibát dob. Új étel definiálásakor megnézi, hogy az ár int-e, volt-e

már ilyen nevű étel, ezek esetén hibát dob, ha minden stimmel, hozzáadja a saját kis listájához, törlénél törli a saját belső listájából. Végül mindig kiírja a listáját a menu.txt-be, törölve ezzel annak eddigi tartalmát, és visszajelez, hogy a módosítás sikeres volt.

Amit még meg szeretnék csinálni:

- Elég idegesítő, hogy hibás opció választása esetén az olyan függvények, amik amúgy meghatározott bemenetre szépen megszakítják magukat, a főmenübe térnek vissza hibás bemenet esetén. A hibás file más, akkor nem kell loopolnia magát, de ha a felhasználó elgépelt valamit, inkább előről kezdhetné csak azt az egy függvényt. Kivittem a **Clear()** függvényt, külön fileba, hogy mindenki hivatkozhasson rá, és ne legyen egymásra hivatkozás, mint ha a főprogramban maradna. Innentől minden ilyen véletlen elírás esetén nem fog majd kelleni mindent előről kezdeni, hanem a függvény meghívja majd saját magát, az esztétika fenntartása miatt pedig töröli majd a konzol addigi tartalmát.
- A módosító függvénypárok önmagukban is elég combosra sikerültek, esetleg ezeket nem lenne nagyon nehéz kirakni külön fájlokba, és sokkal rövidebb lenne az asztalok.py, és az etelek.py is.